Autonomous Code Review & DevOps Agent
DRDO Young Scientists' Laboratory - Smart Materials

Kritanu Chattopadhyay

June 4, 2026

Abstract

Modern software development pipelines demand fast, consistent and intelligent code review at scale. Manual code review processes are time-consuming, prone to inconsistency, and often insufficient in detecting security vulnerabilities, hardcoded secrets, and structural anti-patterns before merging. This project presents a CI/CD Code Review Agent, an automated multi-layered code analysis system that combines traditional static analysis tools with a panel of large language model based agents to produce actionable, gate-driven merge decisions.

The proposed system operates through a sequential analysis pipeline encompassing static security linting, cyclomatic complexity measurement, dependency vulnerability detection, and a language-aware custom rule engine supporting thirteen programming languages. A three-agent LLM ensemble comprising Security, Code Quality, and Performance reviewers independently evaluates submitted code, while a fourth Critic agent arbitrates conflicting findings, eliminates false positives, and assigns a holistic risk grade. Aggregated signals from all the analysis layers are fed into a normalized health scoring mechanism that governs the configurable CI gate thresholds, untimately emitting a PASS, WARN or BLOCK verdict. The system is deployed as a Flask-based serverless action on Vercel with native GitHub pull request integration, enabling fully automated review comment positng on open PRs without manual intervention. The code is available on `https://github.com/kritanuscodingdelight/cicdagent` and hosted on `https://cicdagent-f76t.vercel.app/`

1 Introduction

The rapid growth of collaborative software development has made code quality assurance a critical bottleneck in modern engineering workflows. As codebases scale and teams grow, ensuring that every merged contribution is free from security vulnerabilities, structural weaknesses, and dependency risks becomes increasingly difficult to sustain through manual review alone. Automated tooling that enforces consistent quality gates at the pipeline level while offering explainable and actionable feedback is therefore essential for maintaining long-term software health across diverse technology stacks.

1.1 Motivation and Problem Statement

Manual code review is limited by reviewer availability, domain expertise, and cognitive fatigue. Critical issues such as hardcoded credentials, insecure function usage, and vulnerable dependencies frequently escape detection under time pressure. Existing static analysis tools are language-specific, produce high false-positive rates, and lack the contextual reasoning needed to distinguish genuine risks from benign patterns. No single tool adequately covers the full spectrum of security, maintainability, performance, and dependency hygiene simultaneously. There is therefore a clear need for a unified system that combines deterministic rule-based analysis with large language model reasoning to produce consistent and explainable merge decisions at scale.

1.2 Objectives

The primary objectives are as follows:

- To design and implement a multi-layered automated code review pipeline integrating static analysis, secret detection, dependency automation, dependency auditing, structural inspection, and multi-agent LLM reasoning into a single decision framework.
- To develop a critic-driven arbitration mechanism that reconciles findings across independent reviewers, suppresses false positives, and assigns a calibrated risk grade.
- To expose the pipeline through a REST-API backed by an accessible web interface.
- To deploy the system as a production-grade serverless application with native GitHub pull request integration, enabling automated review feedback directly within the developer workflow.

1.3 Scope of the Project

The project covers automated review of source code submitted via file upload or GitHub pull request URL across thirteen programming languages including Python, JavaScript, TypeScript, Java, C, C++, Kotlin, C#, Ruby, PHP, Swift, Rust, and Go. Python receives the deepest coverage through Bandit linting, Radon complexity measurement, AST-based structural inspection, and pip-audit dependency auditing. All other supported languages are reviewed through regex-based structure detection, a language-aware rule engine, and the LLM agent ensemble. The system is scoped as a review aid within CI/CD workflows and does not perform autonomous correction beyond conservative auto-fixable findings.

2 System Architecture

The CI/CD Code Review Agent is structured as a modular, layered system where each component performs a well-defined analytical responsibility before passing its output downstream to a centralized scoring and decision engine. The architecture is language-agnostic at the orchestration level, with the language-specific modules activated conditionally based on the detected file type. The system is encapsulated within a single Flask application deployed as a serverless function on Vercel, making it stateless and scalable without infrastructure overhead.

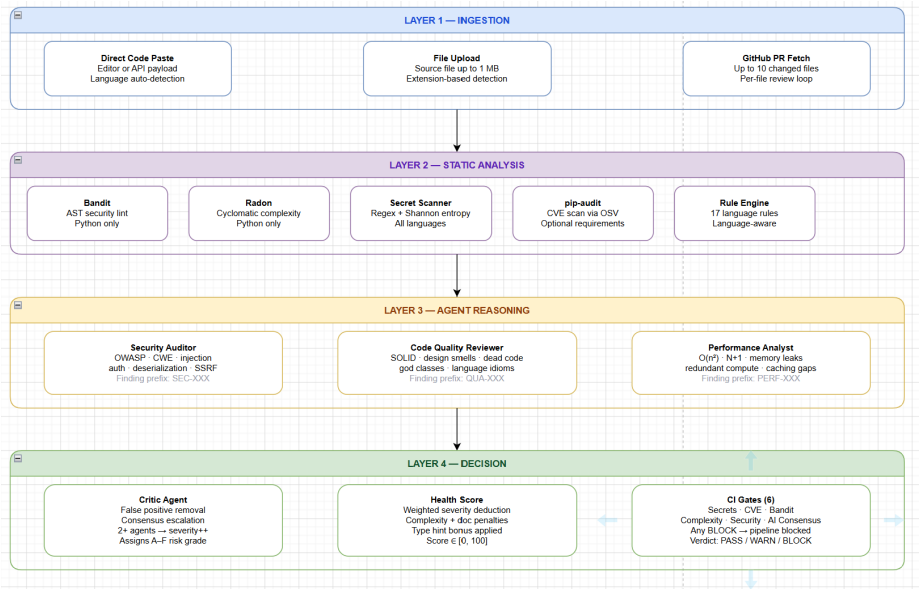


Figure 1: Image showing the system architecture highlighting the four layers - ingestion, static analysis, agent reasoning and decision.

2.1 High-Level Architecture Overview

The system is organized into four logical layers. The *Ingestion Layer* handles code submission through direct paste, file upload or GitHub pull request fetch, normalizing and language-detecting input before passing downstream. The *Static Analysis* layer applies deterministic tools like Bandit, Randon, pip-audit, a custom rule engine, and AST or regex-based structural analysis depending on the submitted language, producing severity-labeled findings without any model inference involved.

The *Agent Reasoning* layer dispatches the submitted code to three independent LLM reviewers covering Security, Code Quality, and Performance perspectives respectively, with a fourth Critic agent arbitrating across all outputs to

suppress false positives and assign an overall A to F risk grade. The *Decision Layer* then aggregates all signals into a normalized health score and evaluates a set of configurable CI gates to emit a final **PASS**, **WARN**, or **BLOCK** verdict surface to the user.

2.2 Component Interaction Flow

Upon receiving a review request, the application loads environment-bound thresholds and model settings, then passes the submitted code simultaneously to the static analysis module and the secret scanner. For Python files, Bandit and Radon run as background subprocesses alongside native AST-based structural inspection. For all other supported languages, regex-based structure detection and the custom rule engine handle the analysis. If a requirements file is provided alongside the code, pip-audit is invoked separately and its CVE findings are merged into the aggregated output.

The consolidated static findings are then formatted into a structured prompt-context and dispatched to the three LLM reviewer agents through the LangChain and Groq integration. Each agent receives the submitted code, the filename, the detected language, and a role-specific instruction set governing its area of evaluation. The individual responses are collected and forwarded to the Critic agent, which produces a final arbitrated findings list with assigned severity levels. The pipeline module receives this complete output, computes the health score using predefined severity weights, evaluates each CI gate independently, and determines the overall verdict, which is rendered in the web interface as gate statuses, a risk grade, and a detailed per-finding breakdown.

3 Core Modules

The core analytical capability of the system is distributed across five independent modules, each responsible for a distinct dimension of code quality assessment. These modules operate deterministically without LLM involvement, forming the foundational analysis layer whose outputs are later combined with agent reasoning to produce the final verdict.

3.1 Static Analysis Engine

The static analysis engine handles the security linting and the complexity measurement of the source files. The security linting is done using Bandit, which scans the abstract syntax tree of the submitted code and flags constructs associated with known vulnerability classes including SQL injection, insecure cryptographic functions, shell injection via subprocess calls, and unsafe deserialization. Each finding is assigned a severity level of LOW, MEDIUM, or HIGH along with a confidence rating, both of which feed into the downstream health scoring mechanism.

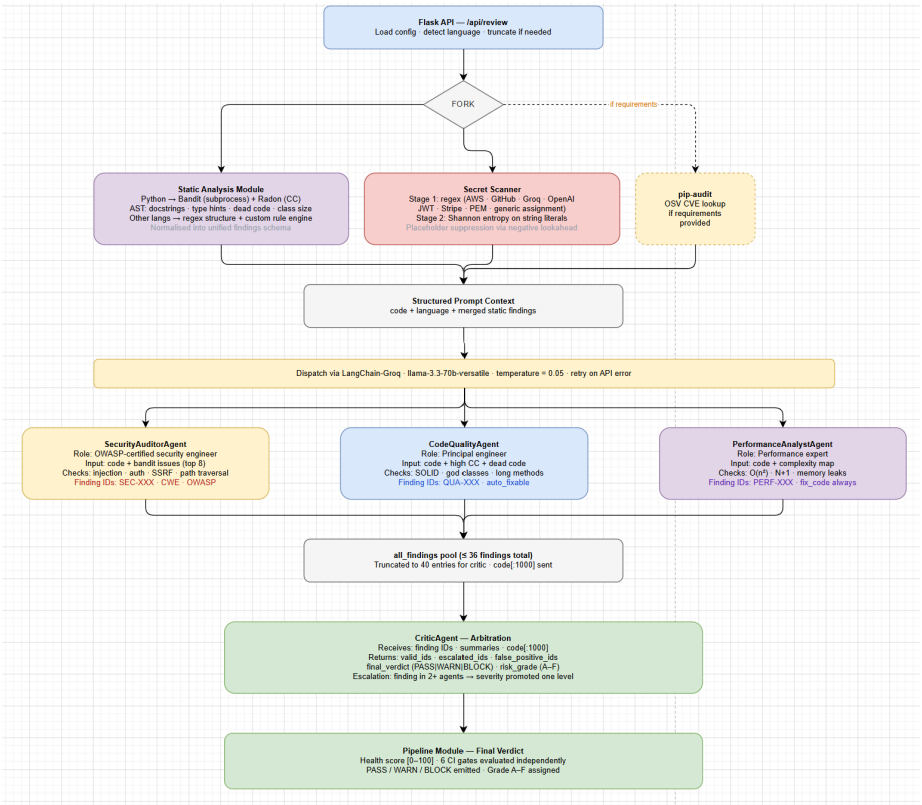


Figure 2: Image highlighting the component interaction flow.

Complexity measurement is handled through Randon, which computes the cyclomatic complexity of each function. Cyclomatic complexity quantifies the number of linearly independent paths through a function's control flow graph, and functions exceeding the configured threshold are flagged as refactoring candidates. Both tools are invoked as background subprocesses and their outputs are normalized into a unified findings schema before being passed downstream.

3.2 Secret Scanner

The secret scanner operates through a two-stage detection mechanism applied uniformly across all supported languages. The first stage applies a curated set of regular expressions targeting known credential formats including AWS access keys, GitHub tokens, Groq and OpenAI API keys, Stripe secret keys, private key PEM blocks, JSON Web Tokens, and generic inline password assignments. Negative lookahead conditions suppress obvious placeholder values, reducing false alarm rates in scaffolded codebases.

The second stage applies Shannon entropy analysis to string literals ex-

tracted from the code. Shannon entropy measures the information density of a string and is defined as the negative sum of the probability of each character multiplied by its base-two logarithm across all unique characters. It is given by:

$$H = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{1}$$

where $p(x_i)$ is the probability of character x_i occuing in the string and n is the number of unique characters.

High-entropy strings matching structural patterns associated with credential formats are escalated as likely secrets even without an explicit regex match, ensuring that both pattern-identifiable and obfuscated credentials are surfaced with high recall.

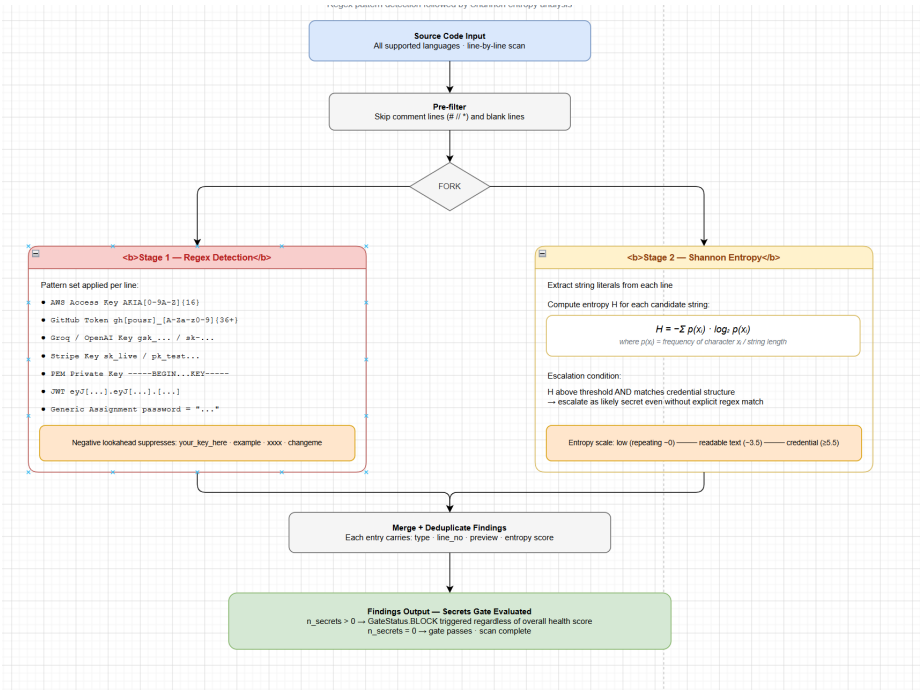


Figure 3: Image highlighting the secret scanner module.

3.3 Dependency Auditor

The dependency auditor accepts an optional requirements file submitted alongside the source code and invokes pip-audit to cross-reference declared package versions against the OSV vulnerability database. Each flagged dependency is returned with its CVE identifier, severity classification, and vulnerability description. Any CVE finding triggers a dedicated CI gate irrespective of the

overall health score, ensuring vulnerable dependencies cannot pass the pipeline unnoticed. In serverless environments where pip-audit may be unavailable, this module degrades gracefully and returns an empty findings list without interrupting the remaining analysis.

3.4 AST and Structural Analysis

For Python files, structural analysis is performed using the native abstract syntax tree parser, which provides a language-spec-compliant code representation without pattern matching. The module traverses the parsed tree to identify functions lacking docstrings, parameters missing type annotations, classes exceeding a configurable line threshold, and unreachable code blocks following unconditional return or raise statements. For all other supported languages, a regex-based fallback performs function boundary detection at a coarser granularity, ensuring structural signals remain available for the agent reasoning layer regardless of the submitted language.

3.5 Custom Rule Engine

The custom rule engine implements language-aware heuristic rules targetting anti-patterns not covered by general-purpose linters. For Python, the engine detects bare except clauses, debug print statements left in the production code, and eval calls on externally sourced input. For C and C++, it flags unsafe standard library functions prone to buffer overflow. For Rust, it identifies unchecked unwrap calls on Result and Option types. For JavaScript and TypeScript, residual console.log statements are flagged as production hygiene violations. Each triggered rule produces a finding with a severity label, a line reference, and a human-readable description passed into the unified findings schema alongside outputs from the other core modules.

4 Multi-Agent LLM Review Pipeline

The multi-agent reasoning pipeline forms the pper analytical tier of the system, operating on top of the deterministic findings produced by the core modules. While static analysis captures rule-violating constructs with precision, it lacks the capacity to reason about intent, context, and holistic code behavior. The LLM agent ensemble addresses this gap by evaluating submitted code from complementary perspectives before a critic-driven arbitration stage consolidates the findings into a single graded decision.

4.1 Agent Design: Security, Code Quality and Performance Reviewers

Each review request is handled by three independent reviewer agents, each guided by a role-specific prompt. The **Security** agent checks for exploitable

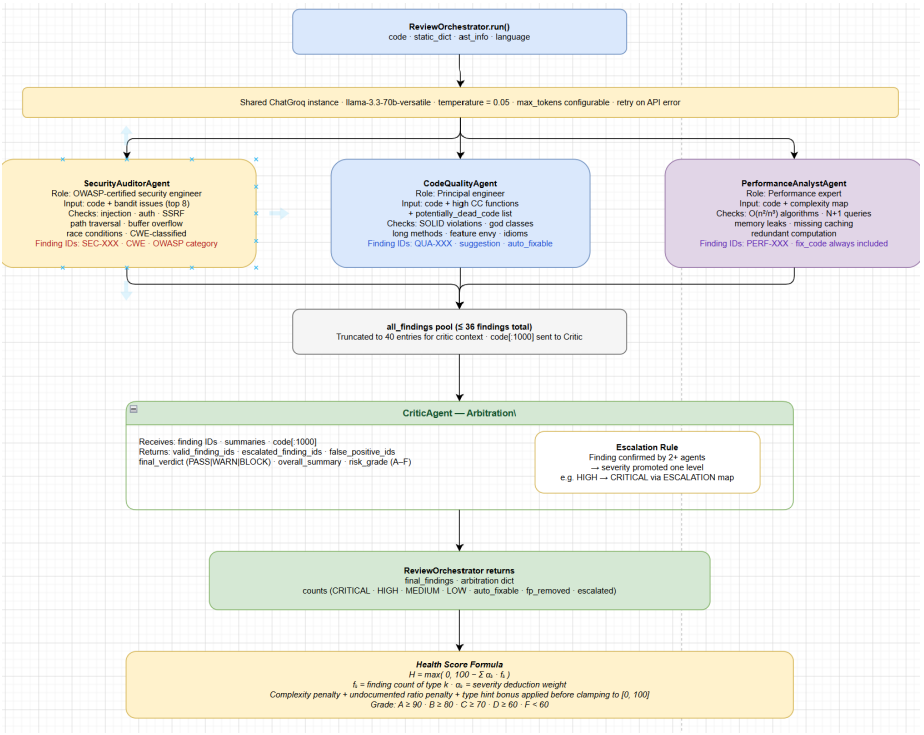


Figure 4: Figure highlighting the multi-agent LLM pipeline showing the coordination between the four agents - Security, Code Quality, Performance Reviewers and Critic Agent

patterns, weak input validation, authentication flaws, and unsafe data handling. The **Code Quality** agent evaluates code structure, adherence to language conventions, naming practices, and maintainability anti-patterns. The **Performance** agent identifies unnecessary computational overhead, inefficient data structure usage, and blocking operations in latency-sensitive environments.

Each agent receives the submitted code, detected programming language, filename, and a structured set of instructions. Responses follow a predefined JSON schema containing findings with a title, description, severity level, line reference, and an indicator of whether the issue can be automatically fixed. The agents are deployed through Groq using the ***Llama 3.3 70B Versatile*** model via LangChain, with configurable retry limits to gracefully handle transient API failures.

4.2 Critic Agent and Arbitration Logic

The Critic agent receives the combined outputs of the three reviewer agents and performs a second-pass evaluation to generate the final findings list. Its

main responsibilities are filtering false positives, identifying consensus findings, and assigning risk levels. Findings reported by multiple agents for the same code region are treated as consensus findings and may be escalated in severity. Findings reported by a single agent undergo a plausibility check before being retained or discarded.

The overall risk grade is assigned on A to F scale based on the weighted aggregation of finding the severities. Considering N_H, N_M, N_L to be the counts of HIGH, MEDIUM and LOW severity findings, the severity score can be computed as:

$$S = w_H \cdot N_H + w_M \cdot N_M + w_L \cdot N_L \tag{2}$$

where $w_H, w_M.$, and w_L are the predefined severity weights. The grade boundaries are applied as monotonic thresholds over S, with grade A corresponding to the lowest severity accumulation and grade F indicating a critically compromised submission.

4.3 Scoring and CI/CD Gate Evaluation

All findings from both the static analysis layer and the agent reasoning layer are aggregated into a unified health score $\mathcal{H}$ normalized to a range of $[0, 100]$, defined as:

$$\mathcal{H} = \max\left(0, 100 - \sum_k \alpha_k \cdot f_k\right) \tag{3}$$

where f_k denotes the count of findings for the type k and $alpha_k$ is the corresponding deduction weight governing the contribution of each finding category to the overall score.

Six CI gates are evaluated independently on the aggregated findings: the **Secrets Gate**, **CVE Gate**, **Bandit Severity Gate**, **Cyclomatic Complexity Gate**, **HIGH Finding Count Gate**, and **Agent Consensus Gate**. Each gate uses a configurable threshold, and a breach in any one of them can escalate the overall verdict from PASS to WARN or from WARN to BLOCK, regardless of the health score. This prevents critical issues from being masked by strong overall performance in other parts of the codebase.

4.4 LLM Backend

The LLM backend is provided by Groq, which enables low-latency inference for large open-weight models. All four agents use the *Llama 3.3 70B Versatile* model, chosen for its strong instruction-following ability and reliable structured outputs. LangChain handles prompt construction, message formatting, and response parsing. Each agent call includes configurable retry logic for transient API failures, falling back to a **WARN** verdict with an explanatory message if all retries fail. Agent responses are also bounded by a maximum token limit to control inference costs while preserving output quality.

5 API Design and Endpoints

The system exposes its functionality through a Flask-based REST API, providing access to code review, file upload, auto-fix, and GitHub pull request integration features. All endpoints use JSON for requests and responses, while errors follow a consistent schema containing a descriptive message and the appropriate HTTP status code.

5.1 Core Review Endpoint

The primary review endpoint accepts a JSON payload containing the source code, with optional filename, language identifier, and requirements file contents. If the language is not provided, it is inferred from the filename extension. The endpoint executes the complete analysis pipeline, including static analysis, secret scanning, dependency auditing, structural inspection, custom rule evaluation, and multi-agent LLM review. It returns a comprehensive result containing the health score, risk grade, CI gate statuses, overall verdict, and a detailed findings list with severity levels and line references.

5.2 File Upload and Language Detection

The file upload endpoint accepts a multipart form submission containing a source file upto 1MB. The system reads the file contents, detecting the programming language from the file extension and returns the detected language identifier alongside the normalized file contents to the client. The endpoint is designed to support the web interface upload flow, allowing the client to pre-populate the cde editor and language selector before invoking the review endpoint.

5.3 Auto-Fix Module

The auto-fix endpoint accepts the original source code and the findings generated by a previous review. It applies conservative fixes only to findings marked as auto-fixable by the agent ensemble. For Python code, the modified source is validated through the native parser before a fix is accepted. Fixes that increase file size beyond a configurable threshold are rejected to avoid unintended structural changes. Instead of returning the rewritten code, the endpoint provides a unified diff between the original and patched versions, allowing developers to review and selectively apply the changes.

5.4 GitHub Pull Request Review

The pull request review endpoint accepts a GitHub PR URL and an optional flag indicating whether the review summary should be posted back to the PR. When a valid GitHub token is available, the system retrieves up to ten modified source files, reviews each independently through the full analysis pipeline, and

aggregates the results. The final verdict is determined by the most severe file-level verdict, meaning a single BLOCK result can block the entire pull request. If comment posting is enabled, a structured Markdown summary containing per-file verdicts, risk grades, and key findings is posted directly to the PR, integrating feedback into the developer workflow without requiring an external dashboard.

5.5 Health and Metadata Endpoints

Two lightweight utility endpoints support monitoring and client configuration. The health endpoint returns the configured model name and a boolean indicating whether a GitHub token is available, allowing clients to verify service readiness before submitting reviews. The languages endpoint provides the list of supported language identifiers and their associated file extensions, enabling client-side validation without hardcoding language mappings in the frontend.

6 Web Interface

The web interface provides a browser-accessible frontend for interacting with the review pipeline without requiring direct API access. It is served as a single HTML template by the Flask application and is designed around a dark visual theme optimized for extended code reading sessions.

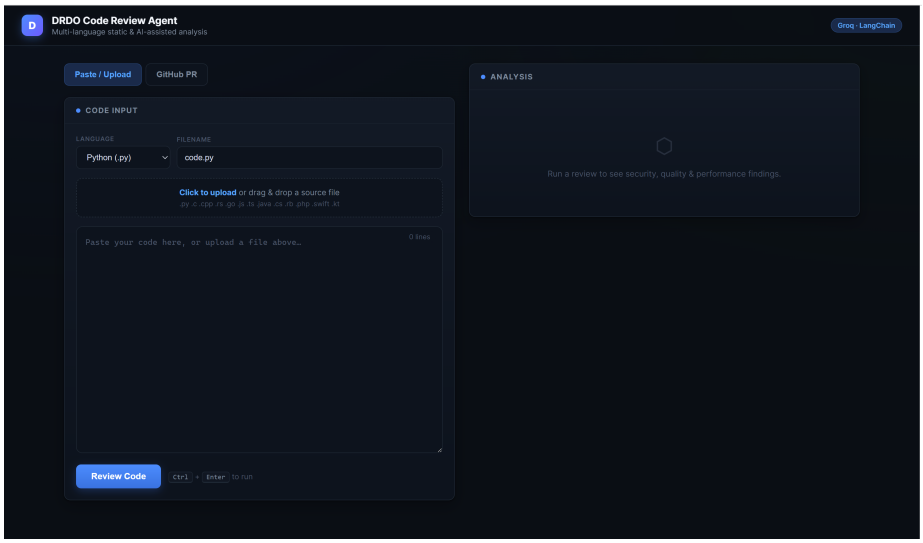


Figure 5: Figure showing the final deployed website UI of the agent, showing the single page, two-panel layout combined with the code upload and the pasting feature along with the GitHub PR Review

6.1 UI Design and Dark Theme

The interface is built as a single-page application with two-panel layout separating code input present in the left from the analysis input on the right. The dark color scheme reduces visual fatigue during prologned use and provides strong contrast for the severity-coded finding labels. The interface is fully responsive and adapts to both desktop and mobile viewpots through fluid layout adjustments.

6.2 Code Submission and Feedback Display

Code can be submitted through three input modes: direct paste into the editor, drag-and-drop file upload, and GitHub pull request URL entry. The language selector is populated dynamically from the languages endpoint and updates automatically on file upload based on the detected language. A line counter below the editor provides real-time feedback on submission size. Upon review completion, the analysis panel populates with the findings list organized by severity, with each entry displaying the finding title, description, affected line reference, and auto-fix availability indicator.

6.3 PASS, WARN and BLOCK Decisions

The verdict is displayed prominently at the top of the analysis panel using color-coded indicators: green for PASS, amber for WARN, and red for BLOCK. Below it, CI gate statuses are shown in a separate grid, with each gate marked as passing or failing along with its evaluated metric value. The Critic-assigned risk grade is presented alongside the health score as a circular progress indicator, providing a quick overview of the submission's overall quality before reviewing individual findings.

7 Deployment

The deployment configuration is minimal by design, leveraging Vercel's native Python runtime support to serve the Flask application without requiring a dedicated server or container orchestration.

7.1 Vercel Serverless Deployment Configuration

The repository includes a Vercel configuration that routes all requests to the Flask application as a single serverless function. To accommodate the latency of multi-agent LLM inference, the maximum execution time is set to 60 seconds. The Python runtime version is pinned to ensure consistent dependency compatibility across deployments.

7.2 Environment Variable Management

All sensitive configuration, including the Groq API key, GitHub token, and thresholds for complexity, severity counts, and token budgets, is managed through Vercel's environment variable system. These settings are loaded at runtime by the configuration module, with safe default values applied where appropriate. For local development, environment configuration files are excluded from version control to ensure that credentials and other sensitive information are not accidentally committed to the repository.

7.3 Degradation in Serverless Runtime

Certain static analysis tools, such as Bandit, Radon, and pip-audit, may be unavailable in the Vercel serverless environment. The system handles these cases gracefully by suppressing tool-specific execution failures and returning empty results for the affected analyses rather than failing the entire review. Core capabilities, including LLM-based agent reasoning and secret scanning, remain fully operational without external binary dependencies, ensuring that a meaningful review result is still produced across all deployment environments.

8 Testing and Validation

System validation was conducted through a combination of functional testing across representative code samples, edge case evaluation, and end-to-end pull request review demonstration on GitHub.

8.1 Sample Code Review Runs

Functional testing involved submitting Python source files containing known vulnerability patterns, high-complexity functions, and dependency requirements with declared CVEs. The system correctly identified and flagged all injected vulnerabilities through the Bandit module, assigned elevated complexity scores to functions with deeply nested control flow, and surfaced CVE findings for packages with known vulnerabilities in the OSV database. Agent-level findings demonstrated consistent alignment with static analysis outputs, with the Critic agent successfully suppressing redundant findings reported independently by multiple reviewers.

8.2 Edge Cases

Edge case testing covered obfuscated secrets encoded as high-entropy string literals without matching any explicit regular expression pattern, confirming that the Shannon entropy stage alone was sufficient to escalate these as findings. Multi-language file submissions with ambiguous extensions were handled through fallback language detection, with the system defaulting to regex-based structural analysis and custom rule evaluation while still dispatching the full

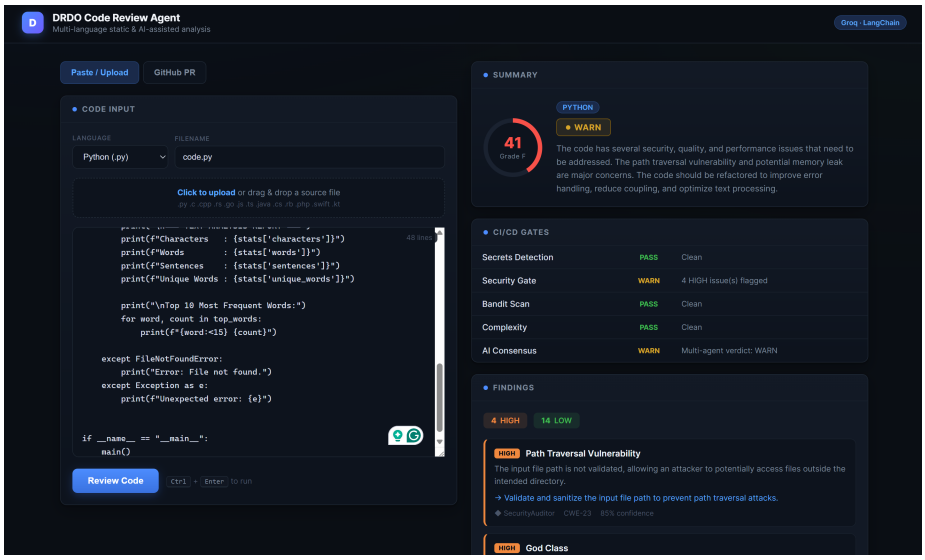


Figure 6: Sample run showing a python script pasted from an external source being analyzed by the agent. It provides detailed analysis of the code, providing information on the CI/CD gates, verdict and the findings.

agent ensemble. Submissions exceeding the LLM context limit were handled through symmetric truncation retaining the beginning and end of the file, preserving function signatures and return statements which carry the highest diagnostic density.

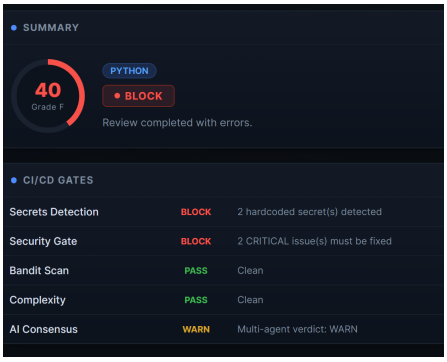
8.3 Pull Request Review Documentation

End-to-end validation of the GitHub integration was performed using a pull request containing multiple modified source files across different languages. The system successfully retrieved the changed files, reviewed each independently, and aggregated the results into a single verdict based on the most severe file-level outcome. The generated Markdown summary was posted as a pull request comment, confirming the correct operation of both the GitHub API integration and the comment-generation workflow in the deployed environment.

9 Limitations and Future Work

Despite its effectiveness, the system has several limitations. LLM-generated outputs may occasionally violate the expected JSON schema. To maintain pipeline reliability, the system employs tolerant parsing and falls back to a WARN verdict when structured extraction fails, although this may result in some findings being discarded. Future versions could leverage constrained decoding or schema-

(a) WARN verdict example



(b) BLOCK verdict example

Figure 7: Representative outputs of the CI/CD Code Review Agent under WARN and BLOCK conditions.

enforced generation to improve output reliability.

In serverless deployments, the availability of external static analysis tools such as Bandit, Radon, and pip-audit may be limited, reducing analysis depth compared to local execution. Containerized deployments with persistent runtimes would provide full tool support while maintaining scalability.

Future enhancements include extending AST-based structural analysis to languages such as Java and TypeScript, incorporating developer feedback to refine agent behavior and reduce false positives, and supporting incremental reviews that analyze only modified code regions to reduce latency and inference costs for large repositories.

10 Conclusion

This project presented the design, implementation, and deployment of an automated CI/CD Code Review Agent that combines deterministic static analysis with multi-agent LLM reasoning to generate gate-driven merge decisions. By integrating Bandit, Radon, pip-audit, entropy-based secret scanning, and language-aware rule evaluation with a four-agent review architecture and critic-based arbitration, the system provides comprehensive and explainable code review feedback across thirteen programming languages. The deployed application demonstrates the practicality of serverless multi-agent review pipelines, offering a scalable and accessible alternative to purely manual or single-tool code review workflows.